

CSE 211: Theory of Computation

Complete Examination Solutions on Turing Machines & Computability Theory

Department of Computer Science and Engineering, BUET

September 11, 2026

Contents

Part I

Examination 2023–2024 (05/05/2026, 22-Batch)

1 Question 9: Binary Equality Comparator

Question 9 (2023–24)

Design a Turing machine (TM) that takes as input two numbers, w_1 and w_2 , in binary, and determines whether the numbers are equal or not[cite: 4]. The tape initially contains w_1cw_2c where ‘ c ’ is a tape symbol that is used as a separator[cite: 4]. Your TM should compare w_1 and w_2 , write 0 after the second c if they are equal, and write 1 after the second c otherwise[cite: 4]. (You can assume that the numbers are of the same length, and use multiple tracks and storage in the state if you wish)[cite: 4].

Full Mathematical Solution & Trace

1. Machine Strategy (2 Tracks & State Storage):

- **Track 1:** Checkmarks (B for unchecked, $*$ for checked)[cite: 2].
- **Track 2:** Data characters ($0, 1, c, B$)[cite: 2].
- **State Storage:** $[q, \text{reg}]$ where $\text{reg} \in \{0, 1, B\}$ stores the bit transported from w_1 to compare against w_2 [cite: 2].

Detailed Algorithmic Flow:

1. **Pick up Bit:** In state $[q_0, B]$, locate the leftmost unchecked symbol $a \in \{0, 1\}$ of w_1 , mark Track 1 with $*$, store a in the finite control, and move right[cite: 2].
2. **Seek w_2 :** In state $[q_1, a]$, move right past the remaining symbols of w_1 and cross the first delimiter c into state $[q_2, a]$ [cite: 2].
3. **Compare Bit:** In state $[q_2, a]$, skip previously checked symbols ($*$) of w_2 [cite: 2]. At the first unchecked symbol $[B, b]$:
 - If $b = a$, mark it checked, clear the register to B , and enter q_{rewind} moving left[cite: 2].
 - If $b \neq a$, a mismatch has been found; move to the blank after the second c , write **1**, and halt[cite: 2].
4. **Rewind:** Move left across w_2 , the separator, and w_1 until the next unchecked position of w_1 can be reached; then return to $[q_0, B]$ [cite: 2].
5. **Equality Verification:** If $[q_0, B]$ encounters the first c , every bit of w_1 has already been matched with the corresponding bit of w_2 . Move to the blank after the second c , write **0**, and halt[cite: 2].

2. Formal 7-Tuple:

$$M = (Q, \Sigma, \Gamma, \delta, [q_0, B], [B, B], \{[q_{\text{halt}}, B]\})[cite: 2]$$

where $\Sigma = \{[B, 0], [B, 1], [B, c]\}$ and $\Gamma = \{B, *\} \times \{0, 1, c, B\}$ [cite: 2].

A more explicit state-set description is

$$Q = \{q_0, q_1, q_2, q_{\text{rewind}}, q_{\text{scan}}, q_{\text{mismatch}}, q_{\text{write0}}, q_{\text{write1}}, q_{\text{halt}}\} \times \{0, 1, B\} \text{[cite: 2]}.$$

3. Transition Function δ : Let $a, b \in \{0, 1\}$.

State	Scanned	Next State	Action
$[q_0, B]$	$[B, a]$	$[q_1, a]$	Write $[*, a]$, Move R (Pick bit from w_1) [cite: 2]
$[q_0, B]$	$[B, c]$	$[q_{\text{write0}}, B]$	Write $[B, c]$, Move R (All bits matched!) [cite: 2]
$[q_1, a]$	$[B, 0/1]$	$[q_1, a]$	Write $[B, 0/1]$, Move R (Scan across w_1) [cite: 2]
$[q_1, a]$	$[B, c]$	$[q_2, a]$	Write $[B, c]$, Move R (Enter w_2) [cite: 2]
$[q_2, a]$	$[*, 0/1]$	$[q_2, a]$	Write $[*, 0/1]$, Move R (Skip checked bits) [cite: 2]
$[q_2, a]$	$[B, a]$	$[q_{\text{rewind}}, B]$	Write $[*, a]$, Move L (Match!) [cite: 2]
$[q_2, a]$	$[B, \bar{a}]$	$[q_{\text{write1}}, B]$	Write $[*, \bar{a}]$, Move R (Mismatch!) [cite: 2]
$[q_{\text{rewind}}, B]$	$[*, 0/1]$ or $[B, \text{any}]$	$[q_{\text{rewind}}, B]$	Move L until reaching left boundary [cite: 2]
$[q_{\text{rewind}}, B]$	(hit left boundary)	$[q_0, B]$	Step R to leftmost unchecked bit [cite: 2]
$[q_{\text{write0}}, B]$	$[B/*, 0/1/c]$	$[q_{\text{write0}}, B]$	Move R past second c [cite: 2]
$[q_{\text{write0}}, B]$	$[B, B]$	$[q_{\text{halt}}, B]$	Write $[B, 0]$, Move R (HALT) [cite: 2]
$[q_{\text{write1}}, B]$	$[B/*, 0/1/c]$	$[q_{\text{write1}}, B]$	Move R past second c [cite: 2]
$[q_{\text{write1}}, B]$	$[B, B]$	$[q_{\text{halt}}, B]$	Write $[B, 1]$, Move R (HALT) [cite: 2]

4. Trace for $w_1 = 10, w_2 = 10$ (10c10c):

$[q_0, B] [B, 1] [B, 0] [B, c] [B, 1] [B, 0] [B, c] \vdash [*, 1] [q_1, 1] [B, 0] [B, c] [B, 1] [B, 0] [B, c]$ [cite: 2]
 $\vdash^* [*, 1] [B, 0] [B, c] [q_2, 1] [B, 1] [B, 0] [B, c] \vdash [*, 1] [B, 0] [q_{\text{rewind}}, B] [B, c] [*, 1] [B, 0] [B, c]$ [cite: 2]
 $\vdash^* [*, 1] [q_0, B] [B, 0] [B, c] [*, 1] [B, 0] [B, c] \vdash [*, 1] [*, 0] [q_1, 0] [B, c] [*, 1] [B, 0] [B, c]$ [cite: 2]
 $\vdash^* [*, 1] [*, 0] [B, c] [*, 1] [q_2, 0] [B, 0] [B, c] \vdash [*, 1] [*, 0] [B, c] [q_{\text{rewind}}, B] [*, 1] [*, 0] [B, c]$ [cite: 2]
 $\vdash^* [*, 1] [*, 0] [q_0, B] [B, c] [*, 1] [*, 0] [B, c] \vdash [*, 1] [*, 0] [B, c] [q_{\text{write0}}, B] [*, 1] [*, 0] [B, c]$ [cite: 2]
 $\vdash^* [*, 1] [*, 0] [B, c] [*, 1] [*, 0] [B, c] [B, 0] [q_{\text{halt}}, B] [B, B]$ (**Appends 0 and Halts**) [cite: 2]

2 Question 10: Multi-tape TM & Single-Tape Simulation

Question 10 (2023–24)

What is a multi-tape Turing machine [cite: 4]? Explain how a one-tape Turing machine with multiple tracks and storage in the state can simulate a multi-tape Turing machine [cite: 4].

Full Mathematical Solution & Trace

1. Definition of Multi-tape TM: A Multi-tape TM consists of a single finite control with k separate, independent physical tapes [cite: 3]. Each tape i has its own independent read/write head [cite: 3]. In one move, based on current state q and the k -tuple of scanned characters $(X_1, \dots, X_k)$, it: 1. Enters a new state p [cite: 3]. 2. Overwrites each cell under each head with Y_i [cite: 3]. 3. Moves each head independently in direction $D_i \in \{L, R, S\}$ [cite: 3].

$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k \text{[cite: 3]}$$

2. Simulation Construction (2k Tracks): Let M be a k -tape TM. We build an equivalent 1-tape TM N whose tape contains $2k$ tracks [cite: 3]:

- **Track $2i$:** Contains the content of tape i of M [cite: 3].
- **Track $2i - 1$:** Contains blanks everywhere except an asterisk (*) indicating the head position of tape i [cite: 3].

3. Simulation of One Move of M :

1. **Scan Phase:** N 's single head sweeps from the leftmost head marker to the rightmost head marker[cite: 3]. N 's finite control stores a tuple $[q, X_1, X_2, \dots, X_k, \text{count}]$, recording the symbol X_i beneath each marker $*$, and counting markers so it knows when all k symbols are collected[cite: 3].
2. **Update Phase:** N determines M 's move: $\delta(q, X_1, \dots, X_k) = (p, [Y_1, \dots, Y_k], [D_1, \dots, D_k])$ [cite: 3]. N sweeps back across the tracks, rewrites each cell X_i with Y_i , and shifts the head marker $*$ on Track $2i - 1$ left or right according to D_i [cite: 3].
3. **State Update:** N changes its internal control state to p [cite: 3].

Time Complexity: After n moves, the markers are separated by at most $2n$ cells[cite: 3]. The sweep takes $O(n)$ steps per move[cite: 3]. Simulating n steps takes $\sum_{i=1}^n O(i) = O(n^2)$ time on a single tape[cite: 3].

3 Question 11: Non-RE Language & Halting Problem Undecidability

Question 11 (2023–24)

Provide an example of a language which is not recursively enumerable[cite: 4]. Devise a proof to show that the Halting problem is not recursive[cite: 4]. (You can assume that the Universal language is known to be not recursive)[cite: 4].

Full Mathematical Solution & Trace

1. **Example of Non-RE Language:** The **Diagonalization Language** (L_d) is not RE[cite: 5]:

$$L_d = \{w_i \mid w_i \notin L(M_i)\}$$

where w_i is the i -th binary string and M_i is the i -th Turing machine in lexicographical order.

2. **Proof that Halting Problem (H_{TM}) is Not Recursive:** Assume $L_u = \{\langle M, w \rangle \mid M \text{ accepts } w\}$ is known to be non-recursive (undecidable)[cite: 4]. We define the Halting Problem:

$$H_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ halts on input } w\}$$

Reduction ($L_u \leq_m H_{\text{TM}}$): Suppose for contradiction that H_{TM} is recursive[cite: 4]. Then there exists a decider R such that:

$$R(\langle M, w \rangle) = \begin{cases} \text{ACCEPT} & \text{if } M \text{ halts on } w \\ \text{REJECT} & \text{if } M \text{ loops on } w \end{cases}$$

We construct a decider D for L_u using R as a subroutine:

1. On input $\langle M, w \rangle$, run decider R on $\langle M, w \rangle$.
2. If R rejects, then M loops forever on w ; therefore M cannot accept w . Decider D **REJECTS**.
3. If R accepts, then M is guaranteed to halt on w . Run the universal simulation of M on w until it terminates.
4. If M enters an accept state, D **ACCEPTS**; otherwise, D **REJECTS**.

Decider D halts on all inputs and decides L_u , contradicting that L_u is non-recursive[cite: 4]. Thus, the Halting Problem is not recursive (undecidable)[cite: 4].

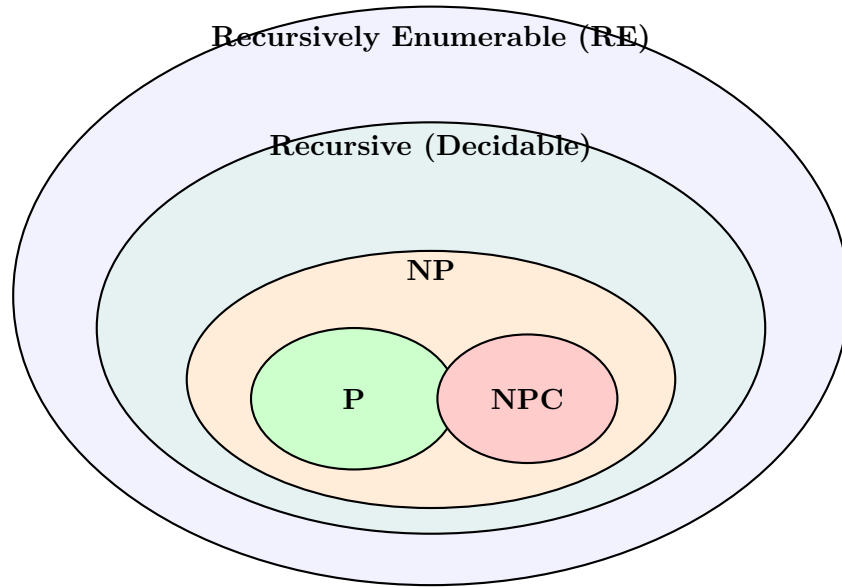
4 Question 12: Complexity Hierarchy & PSPACE vs. NPSPACE

Question 12 (2023–24)

- Draw a Venn diagram showing the widely believed relationship among the classes of languages P, NP, NP-complete, recursive, and recursively enumerable[cite: 4].
- What is the relationship between the classes PSPACE and NPSPACE[cite: 4]? Justify your answer briefly[cite: 4].

Full Mathematical Solution & Trace

1. Venn Diagram of Complexity Classes:



2. Relationship between PSPACE and NPSPACE:

$$\mathbf{PSPACE = NPSPACE}$$

Justification (Savitch's Theorem): Savitch's Theorem states that for any space-constructible function $f(n) \geq \log n$, $\text{NSPACE}(f(n)) \subseteq \text{DSPACE}((f(n))^2)$. For polynomial space, $f(n) = n^k$. Therefore:

$$\text{NPSPACE} \subseteq \text{DSPACE}((n^k)^2) = \text{DSPACE}(n^{2k}) \subseteq \text{PSPACE}$$

Since deterministic space is trivially a subset of nondeterministic space ($\text{PSPACE} \subseteq \text{NPSPACE}$), they are identical: **PSPACE = NPSPACE**.